PalletDiT: Pallet Pose from Forkview Images with a Pixel-Space Diffusion Transformer

Manuj Naman, with Claude (Anthropic) as implementation assistant
Anantak Robotics
manujnaman@anantak.com

PalletDiT adapts PointDiT to predict metric, pallet-frame geometry and a per-pixel class from a single forklift camera image, and recovers the pallet pose by pixel voting, single-frame or warm-started along the approach. Working paper, status of 2026-10-20 evening. Code: branch pallet-pose, package pallet/.

Abstract

A forklift approaching a pallet needs the pallet's pose relative to the camera from several metres away, continuously, from a single low-resolution RGB camera. We adapt PointDiT, a pixel-space diffusion transformer for monocular point maps, to this task. Instead of a per-frame normalised point map in the camera frame, the network predicts for every pixel seven quantities in a metric, pallet-anchored frame: a four-way class (pallet, load, ground, background), the ray length, the hit point in the pallet frame, and two ray angles. The targets come from a lidar reconstruction of each recorded pick, rendered into the camera. The backbone, the frozen DINOv3 conditioning and the rectified-flow objective of PointDiT are kept; only the input/output channel count, the per-channel masking and an added classification head change. Each pallet or load pixel then casts a vote for the camera centre in the pallet frame; the median vote is the pose estimate. Fine-tuned from the public PointDiT-B/256 weights in six stages up to 2,030 picks (62k images), the model reaches a median pallet-origin error of 5.5 cm within 2 m, 7.7 cm at 2–3 m and 11.2 cm at 3–4.5 m on 201 picks that no stage trained on, with a rotation error of about 1 degree, in a single network evaluation at 432x240 pixels.

I. Problem

The camera is the forkview camera of a forklift (424x240 raw frames, rotated upright to 432x240 after padding; pinhole, f ~317–319 px, no distortion). During an approach the robot records 20–50 images from ~11 m down to ~1.2 m. For each image we want the rigid transform `pallet_P_cam` (X_pallet = R X_cam + p) between the camera and a frame anchored on the pallet: origin at the centre of the pallet's front face on the ground, x lateral, **y down** (gravity is exactly +y), z into the pallet.

Why not regress the pose directly? A dense, per-pixel formulation (i) uses the whole visible pallet and load surface instead of a few keypoints, (ii) degrades gracefully with occlusion and distance, (iii) gives a segmentation and a metric depth map for free, and (iv) lets later stages fuse evidence over the approach sequence. PointDiT already produces dense geometry from one image with strong priors from DINOv3; the question was whether it transfers to a metric, object-anchored frame with a small dataset.

II. Background: PointDiT in one paragraph

PointDiT (Xu et al., 2026) is a plain ViT that denoises raw 3D point-map patches in pixel space, conditioned on image tokens from a frozen DINOv3 encoder (four intermediate layers concatenated), with image and point tokens concatenated along channels and projected to the model width. It is trained with rectified flow, x-prediction and a logit-normal timestep distribution; sampling is deterministic Euler from

zero noise (1 step for the 256-px models). Targets are camera-frame XYZ maps centred on their valid mean and divided by the mean distance, i.e. per-frame affine normalisation; the loss is a masked per-pixel MSE in v-space. The B/16 model has 12 blocks, width 768, patch 16, a bottleneck patch embedding (conv 16x16 to 128 then linear to 768) and a final adaLN-modulated linear that emits `16*16*C` values per token, unpatchified to full resolution.

III. Method

3.1 Targets: seven channels in the pallet frame

For each pixel of the upright image, from the lidar reconstruction (SensorUnit pipeline, `pallet_rays.py`):

channel	symbol	definition	where defined
class	c	0 pallet, 1 load, 2 ground, 3 background	every pixel
range	r	length of the ray from the camera centre to what it hits (m)	hit with known range
point	P = (px, py, pz)	the hit point in the pallet frame (m)	hit
theta	θ	angle between the ray and gravity-down (+y of the pallet frame)	every pixel
phi	φ	atan2(d_z, d_x) of the ray projected on the ground plane; a ray straight into the face has φ = +π/2	every pixel

With the unit ray d_cam of a pixel (from the intrinsics), R the camera-to-pallet rotation and c the camera centre in the pallet frame, the pallet-frame ray is d = R d_cam and

```
P = c + r d,    θ = arccos(d_y),    φ = atan2(d_z, d_x).
```

θ and φ depend only on the camera pose, not on the scene; they are included as a dense, image-independent handle on the rotation. On the real data, P reconstructed from (r, d, pose) matches the stored P to under 1 mm for pallet, load and ground pixels; background pixels are splatted lidar points up to ~16 cm off the ray and are never used for pose.

Fixed encoding. PointDiT's per-frame centring and scaling destroy metric meaning, so they are removed. Instead each channel gets one fixed, invertible affine chosen once from the data: offsets (0, 0, 0, 0, π/2, π/2) and scales (2, 2, 4, 4, 0.5, 0.5) metres or radians, so the network sees O(1) numbers (section 5 of the code guide). Range and P are supervised only up to 12 m.

Padding. 424 is not a multiple of the 16-px patch; 4 black rows are added above and below (432x240), the principal point shifts by 4 px, padded pixels carry class 255 (ignore) and all-false masks. Width 240 is already a multiple of 16.

3.2 Masking

Three masks travel with every sample:

- `pixel_mask` [H,W]: real (non-padded) pixels.
- `valid_mask` [6,H,W]: per channel. P and range are valid where the range is known (and ≤ 12 m); θ and φ are valid on every real pixel. This replaces PointDiT's single [H,W] mask repeated over three channels.

- `cls` [H,W] with 255 on padding: the cross-entropy ignore index.

The flow loss is averaged over the true entries of `valid_mask` only; invalid target entries are replaced by noise before being mixed into the diffusion input (PointDiT's `--noise_fill_invalid`), so the network cannot read "zero means invalid".

No horizontal flip is used (it would mirror the pallet frame); no per-frame normalisation; colour jitter and random crop are available but the baseline trains at the full 432x240 without augmentation.

3.3 Model changes

The PointDiT module is used unchanged. It is constructed with `in_channels = 6` (the continuous targets) and its final linear is widened after construction to emit `6 + 4` channels: the first six are the x-prediction of the continuous targets, the last four are class logits produced by the same final layer. The patch-embedding conv (`[128, 3, 16, 16]` pretrained) and the final linear are the only weights that do not load from the public checkpoint; everything else (all transformer blocks, the concat projection, positional embeddings, the adaLN of the final layer) is initialised from PointDiT-B/256. DINOv3-B/16 stays frozen. The positional embedding is bicubically resized from the 16x16 pretraining grid to 27x15 patches on the fly.

3.4 Objective

Per sample, with x the encoded 6-channel target and e ~ N(0, I):

```
t ~ sigmoid(N(-0.8, 0.8)), with probability 0.1 set t = 0
z = t * x_filled + (1 - t) * e            (x_filled = x where valid, e elsewhere)
x_hat, logits = net(z, t, image)
L_flow = mean over valid entries of  (x_hat - x)^2 / max(1 - t, 0.05)^2
L_cls  = cross-entropy(logits, c, ignore 255)
L = L_flow + L_cls
```

The weighting `1/(1-t)^2` is exactly PointDiT's v-space MSE rewritten for x-prediction, so the objective and the pretrained weights agree. The 10 % of samples at t = 0 train the network under the exact condition used at inference. The class logits are supervised at every t.

3.5 Inference and pose voting

Inference is PointDiT's deterministic Euler integration from z = 0; with one step the network is a plain regressor `x_hat = net(0, t=0, image)`, costing one forward pass with cached DINOv3 features.

Pose from the dense prediction:

1. **Rotation.** Every real pixel's predicted (θ, φ) gives the ray in the pallet frame, d_pal = (sin θ cos φ, cos θ, sin θ sin φ); its camera ray d_cam is known. R is the Kabsch solution of min Σ |d_pal − R d_cam|² over all real pixels (~100k pairs).
2. **Translation.** Each pixel classified pallet or load votes for the camera centre: c_i = P_i − r_i R d_i. The estimate is the coordinate-wise median of the votes; the median absolute deviation of the votes ("spread") is a confidence proxy.

The pallet origin in the camera frame is $-R^{\top} c$. The metric reported throughout is |c_est − c_gt| in the pallet frame, binned by the distance of the camera to the pallet face.

IV. Data

- **Source.** Recorded picks of two forklifts (robots 249 and 250) at one site. The SensorUnit pipeline reconstructs each pick from the XT32 lidar and the carriage odometry, fits the pallet and load as 5 cm oriented patches, calibrates the camera per pick (focal length and rotation by edge reprojection), and renders the per-pixel targets by exact ray–patch intersection plus a splat of the remaining lidar points and the analytic ground plane. Box reprojection agrees with the pipeline's own portrait corners to 0.05 px.
- **Scale.** 2,895 picks exist; by 2026-10-16 07:30, 1,820 had rendered targets (about 56k images), produced on two machines in parallel: ana-ml-02 builds the full history in ascending order, the laptop builds newest-first from the same raw episodes.
- **Training store.** Images and *compact* target records (class map, range, pose; P, θ, φ are rebuilt exactly on load) live in `~/Anantak/Data/PalletTrainImages/<robot>/<pick>/`, about 250 KB per image instead of 1.4 MB. A manifest (one row per image) is built from the store.
- **Splits.** By pick, never by image (an approach sequence is highly correlated), stratified by the human-reviewed pallet kind where known, 20 % of picks held out; validation uses an even 600-image subsample of the held-out images.

V. Experiments

All runs: PointDiT-B/16 at 432x240, frozen DINOv3-B/16, bf16, AdamW (0.9, 0.95), cosine schedule, EMA, grad-clip 1.0, 1-step inference, one 16 GB laptop GPU (RTX 3080 Ti). Medians over the validation images per face-distance bin, pallet-origin error in cm.

run	init	train picks / images	epochs, batch, lr	val picks	0–2 m	2–3 m	3–4.5 m	4.5–6 m	6–8 m	all	P err	rot (deg)	pallet IoU
1 baseline	B/256 public	30 / 1,061	60, 8, 1e-4	8	7.2	11.0	16.0	25.4	34.4	17.1	14.0	0.9	0.65
2	run 1	318 / 9,809	12, 16, 1e-4	80	7.7	9.3	12.5	21.2	27.1	12.2	11.2	1.0	0.68
3	run 2	655 / 20,151	12, 16, 7e-5	164	6.4	8.9	11.7	19.2	21.9	11.4	9.5	1.0	0.69
4	run 3	816 / 25,049	10, 16, 5e-5	204	5.2	7.5	11.3	14.5	21.7	9.7	8.9	0.9	0.72
5	run 4	1,453 / 44,340	8, 16, 4e-5	359	6.1	6.9	11.5	13.6	20.2	10.5	8.5	0.9	0.71
6	run 5	2,030 / 62,090	8, 16, 3e-5	468	5.7	7.0	10.5	15.4	16.8	10.2	8.3	1.0	0.71

Class accuracy is 0.98–0.99 throughout; load IoU 0.90–0.92; the ground and background classes are easy. The validation set changes with every run (more held-out picks), so rows are indicative rather than strictly comparable; the trend with data is nevertheless monotone, and within each run the error curve flattens after 6–8 epochs.

Clean comparison. Because each run re-split a larger store, a later run has trained on part of an earlier run's validation picks, so cross-evaluating on another run's split is biased (run 6 on run 5's split gives 9.2 cm, flattering it by ~1 cm). The table below evaluates runs 3–6 on 201 picks (5,854 images, 600 used) that

no run ever trained on; they are 84 % robot 250 and harder than the per-run splits (run 6 scores 11.1 here against 10.2 on its own split).

clean held-out, 201 picks	0–2 m	2–3 m	3–4.5 m	4.5–6 m	6–8 m	8–12 m	all	P err	with GT mask + R	rot (deg)
run 3 (655 picks)	6.9	10.4	12.8	19.5	25.1	38.2	12.5	10.2	9.3	1.2
run 4 (816)	6.6	9.5	12.1	18.2	22.2	35.8	12.0	9.5	8.2	1.2
run 5 (1,453)	6.1	8.9	11.4	15.7	21.7	28.6	11.1	8.6	7.3	1.1
run 6 (2,030)	5.5	7.7	11.2	16.5	21.2	33.6	11.1	8.3	6.7	1.1

Images per bin: 133, 137, 104, 93, 87, 46. On this set the near field improves steadily with data (0–2 m 6.9 to 5.5 cm, 2–3 m 10.4 to 7.7 cm, P error 10.2 to 8.3 cm, GT-mask floor 9.3 to 6.7 cm), the 3–4.5 m bin is flat after run 5, and the far bins are too few and too noisy to rank runs 5 and 6 (46–93 images, medians moving in both directions). The overall median is equal for runs 5 and 6 because it is dominated by the mid and far bins.

Where the remaining error is. With the ground-truth class mask and ground-truth rotation but the *predicted* P and range, the origin error of run 4 drops to 4.1 / 5.6 / 8.2 cm in the first three bins: mask and rotation together cost 1–3 cm. The vote spread is 4–5 cm near the pallet. Within 3 m the 90th percentile is ~16 cm and 3 % of images exceed 20 cm; the error is mostly lateral and depth (~4.5 cm each), height is 1.3 cm. Far bins are limited by pixel resolution: at 8 m a pallet spans ~45 px.

Sampling steps. Evaluating the run-4 weights with 3 Euler steps instead of 1 lowers the P error from 8.9 to 7.4 cm and the vote spread from 4.9 to 3.3 cm, but the angles get worse (θ error 0.22 to 0.33 deg), and so does the rotation; the origin error improves at 2–4.5 m (7.5 to 6.7, 11.3 to 8.9 cm) and worsens beyond 4.5 m (14.5 to 18.5 cm), a wash overall. A hybrid (angles from 1 step, P and range from 3 steps) behaves the same. Rotation from pallet/load pixels only is worse (1.26 vs 0.91 deg): the ground and walls carry most of the rotation signal.

Cost. Training throughput 18–20 images/s at batch 16 (8.6 GB); inference is one forward pass. Fine-tuning from the public checkpoint converges in a few thousand steps; training from scratch was not attempted.

VI. Discussion

- The per-frame normalisation of PointDiT is not needed for transfer: fixed per-channel scales and the pretrained blocks suffice, and the model keeps metric accuracy at the few-centimetre level near the pallet.
- The classification head shares the final layer with the regression; this cost nothing in accuracy and avoids a second decoder.
- Data still helps, slowly. On the clean held-out set the near-field error fell by 0.3–0.6 cm per stage from run 3 to run 6 while the training set grew 3x; the mid field stopped improving after run 5 and the far field cannot be ranked with 46–93 images per bin. The earlier "saturation" reading from per-run splits was an artefact of split drift. Another doubling of same-site data would buy perhaps half a centimetre near the pallet; the levers below are cheaper.

Error budget

Reading run 6's clean held-out columns gives a rough decomposition of the near-field origin error (0–3 m, 5.5–7.7 cm):

component	evidence	size
vote dispersion of P and range	median |vote − median| ("spread")	3.7–3.9 cm per vote, largely averaged out by the median over 10^4 votes
class mask (which pixels vote) and rotation	origin error with GT mask + GT R vs predicted	1.5–2.5 cm
pallet-frame bias of P	residual with GT mask + GT R	4–5.5 cm
rotation alone	θ error 0.2 deg, φ 0.8 deg, R error 0.9 deg	at 2 m a 0.9 deg error moves the centre by ~3 cm

So the floor is set by a *systematic* P/range bias per image rather than by noise: the median of 20k votes is 4–5 cm off even with a perfect mask and rotation. Candidate causes, in the order they should be checked: (a) the camera calibration per pick (focal length and rotation are fitted per pick from edge reprojection; a 1 % focal error at 2 m is 2 cm of depth), (b) the timestamp interpolation of the carriage pose to the image time during motion, (c) label bias from the 5 cm patch quantisation, (d) a true model bias (e.g. systematically shrinking the pallet). (a)–(c) are target errors and would also appear as a residual when voting with the *ground-truth* P; that single experiment separates label noise from model error and should be run first.

Beyond 4.5 m the error grows roughly linearly with distance (14 cm at 5 m, 21 cm at 7 m, 27 cm at 10 m): a pallet spans ~90 px at 4 m and ~45 px at 8 m, so a one-pixel error in P is 2–4 cm there, and the class mask erodes (pallet IoU 0.75 near, 0.65 far).

Next steps, with expected gain and cost

1. **Diagnose the bias (no GPU, hours).** Vote with GT P/range/angles and the *predicted* mask, and with predicted P and the GT mask, on the saved validation predictions; plot the signed per-axis error against face distance and against the pick's camera-calibration residual. Outcome: whether 4–5 cm is label or model. If it is label, better calibration or a calibration channel in the network (predict a per-image focal correction) is the fix, not a bigger model.
2. **Voting and confidence (CPU, a day).** Inlier selection (e.g. iterate the median with a 3-sigma cut on |vote − c|), a per-pixel confidence from the class softmax or from the agreement between the 1-step and 3-step predictions (the hybrid showed the two differ most where P is unreliable), and weighting votes by predicted range (near pixels are more accurate). Expected gain: 1–2 cm near, more far, by removing the mask and dispersion terms; cheap to evaluate on the same JSON/predictions.
3. **Sequence fusion (CPU, days; the original step 4).** Chain the per-image votes with the carriage odometry into a filter over the approach; the pose is then estimated from 20–50 images rather than one. Noise terms average out; the systematic bias of (1) does not, which is why (1) comes first. Also yields the velocity and a confidence for the unit's planner.
4. **Model capacity: PointDiT-L/256 (GPU, ~1 day).** Same code path, public weights, 24 blocks / width 1024, ~3x compute, batch 8 at 432x240 in 16 GB. Expected: sharper P and masks, mainly in the middle bins; the paper's own scaling (B to L) is worth ~10 % relative on its benchmarks. Cheap to try because the data and the pipeline are fixed.
5. **Resolution (GPU, 4x cost).** Upsample the input 2x (864x480, 54x30 patches); DINOv3 and the ViT scale quadratically in tokens, so training throughput drops to ~5 img/s and inference to ~4x the current cost on the unit. Targets the far bins only; run after (4) and only if the far field matters for the product.
6. **Multi-step inference** is already measured: 3 steps help P (−1.5 cm) and the 2–4.5 m band but hurt the angles; it becomes useful once the rotation is taken from a better source (sequence fusion or

odometry), at 3x inference cost.
7. **Not yet addressed:** other sites and robots (domain shift), lighting extremes, overhanging loads, and the TensorRT export (the DINOv3 encoder already runs on the unit; the ViT is plain attention and should export directly).

The recommended order is 1 → 2 → 4 in parallel with 3, then decide on 5 from the product requirement for the far field.

VII. Sequential estimation along the approach

Idea. During an approach the camera takes 20–50 images of the same static scene. Because the targets live in the pallet frame, the previous frame's dense estimate is also a prediction for the next frame: P of a scene point does not change, and theta/phi follow exactly from the propagated pose. Because the network was trained with the flow objective at every t, it can be *warm-started*: instead of integrating from z = 0 at t = 0, start at time t0 from the propagated prior, z0 = t0 x_prior (+ noise where the prior is unknown), and let the image correct it. The sequence then refines one estimate rather than re-estimating from scratch, and the pose can be fused over frames.

Prior construction (`pallet/sequence.py`). Propagate the previous pose with the inter-frame motion dT (from odometry; here the recorded poses, i.e. perfect odometry): $R' = R A^{\top}$, $c' = c - R' b$. Angles: exact for every pixel from R'. P and range: the previous frame's pallet/load pixels re-rendered into the new view with a 3-px z-buffer splat (P unchanged, range recomputed), plus the ground plane y = 0 analytically where it is visible (in front of the face or beside the pallet). Unknown elsewhere.

Findings on 30 held-out picks, 839 frames, one warm step (`runs/seq_eval*`; origin error, cm, median per bin):

prior	0–2 m	2–3 m	3–4.5 m	4.5–6 m	6–8 m	8–12 m	all	rot (deg)
none (single frame, run 6)	5.6	8.2	12.2	15.4	21.6	26.8	10.9	1.15
previous GT maps + GT pose, t0 0.5	3.6	3.4	3.7	3.8	4.4	6.4	3.8	0.25
own maps + GT pose, t0 0.5	4.8	6.4	7.3	7.1	5.4	6.6	6.0	0.41
own maps + GT pose, maps only (no angle prior)	6.9	10.7	8.3	12.0	19.8	27.8	10.8	0.96
own maps + own smoothed pose, t0 0.5	8.6	11.7	15.4	17.0	28.5	33.8	13.3	1.21

1. The mechanism works: given a correct pose, the network refines its own previous maps to 6.0 cm overall, and the far bins collapse from 22–27 cm to 5–7 cm. Three Euler steps from the prior are worse than one everywhere.
2. Most of the gain enters through the angle channels: with a correct pose they are exact, the network's rotation error drops from 1.15 to 0.4 deg, and every vote improves. A maps-only prior gives almost nothing.
3. With the model's own pose there is no gain and a slight loss, because the network copies the ~1.2 deg rotation error back in through the same channels, and the error is systematic per pick (smoothing over frames does not reduce it).
4. Two inference details mattered: unknown prior entries must be filled with noise, as in training (zeros mean "a point at the pallet origin"), and the network never learned to predict a channel from the image where its input is noise at intermediate t, so partial priors damage uncovered pixels. A by-

product: the predicted P and range disagree along the ray by ~15 cm median, which argues for a consistency term in training.

Prior-conditioned training. The network must be taught to *correct* a prior with a realistic pose error rather than copy it. Run 6 was fine-tuned for 3 epochs with, for half of the samples, the input noised from a prior rendered on the fly from a frame 1–3 back in the same pick (GT pallet/load pixels + visible ground), with the pose perturbed by N(0, 1 deg) / N(0, 5 cm), 4 cm P noise, a 4 cm per-frame P bias and 20 % block dropout; the loss is unchanged and covers every valid pixel (`runs/b256_prior1`). On the same 30 picks:

prior1, one warm step	0–2 m	2–3 m	3–4.5 m	4.5–6 m	6–8 m	8–12 m	all	jitter	rot (deg)
none (single frame)	5.4	7.6	10.7	13.7	23.3	27.2	10.2	5.4	1.19
own maps + own smoothed pose, t0 0.5	5.3	7.7	10.5	13.9	25.0	27.3	9.5	4.2	1.11
own maps + own smoothed pose, t0 0.8	5.3	8.2	9.5	13.5	22.2	26.5	9.5	3.7	1.14
own maps + GT pose, t0 0.8	4.0	4.9	5.6	7.2	8.2	9.7	5.5	4.0	0.42
previous GT maps + GT pose, t0 0.8	3.2	3.6	3.8	4.3	5.5	7.8	4.1	3.0	0.26

A second variant (`prior2`) added a stronger pose corruption (2 deg / 8 cm) and an **on-ray consistency loss**, the mean distance between the predicted P and the point at the predicted range along the pixel ray in the pallet frame, weight 0.5 (motivated by the 15 cm disagreement noted above). After two epochs, on the clean 201-pick split, it lowers the origin error from 10.5 (run 6) to 8.4 cm overall and from 5.8 / 7.9 / 11.1 to 4.3 / 5.8 / 8.5 cm in the first three bins, with P error and rotation unchanged (7.7 cm, 1.0 deg) and the GT-mask floor down from 6.8 to 4.7 cm: the votes became mutually consistent, which is what the median needs. It is the best single-frame model to date. In sequence mode its own-pose warm start gives 8.0–8.3 cm against 8.4 single-frame (jitter 4.1 to 2.8 cm); with a correct pose 4.4 cm; with the previous frame's ground truth 3.2 cm.

Three effects of prior-conditioned training: (i) the single-frame model itself improved (run 6 10.5 to prior1 9.9 to prior2 8.4 cm on the clean split), partly because the prior samples act as a strong augmentation and partly through the consistency term; (ii) with its own pose the warm start now helps instead of hurting (13.3 to 9.5 cm, jitter 5.4 to 3.7) because the network no longer copies the pose error; (iii) the gain stays small because the rotation error does not move (1.1–1.2 deg): it is systematic per pick, the angle prior reproduces it, and the image evidence at one step does not override it. With a correct pose the same network reaches 5.5 cm. **The rotation is now the whole game**: a per-pick rotation error of ~1 deg at 3 m is 5 cm of origin error, which is most of the gap between 9.5 and 5.5 cm.

Anatomy of the rotation error (run 6, clean split, 177 picks): the per-pick mean rotation error is 0.95 deg (median) against a frame-to-frame scatter of 0.64 deg; the global mean is 0.16 deg; across picks the mean error varies by 1.35 deg about the vertical axis and only 0.3–0.4 deg about the other two. It is a **per-pick yaw bias** of the pallet heading. Two attempts to recover it from the sequence failed by construction: the consistency of the voted camera trajectory with the odometry is invariant to a global yaw (both trajectories rotate together; fitted yaw uncorrelated with the measured bias, r = 0.08), and the dispersion of the votes is equally blind (r = 0.05), because the network's P and angles are mutually consistent even when both carry the bias. So the bias is either in the labels, a per-pick heading error of the lidar pallet fit, or a perception bias that moves P and the angles together. The two are separable without an external reference: **different models would not share a perception bias, but every model disagrees with a biased label in the same**

way. On the 177 clean held-out picks, the per-pick mean yaw error of runs 3, 4, 5, 6 and the prior-trained model are pairwise correlated at r = 0.92–0.99 (spread 1.3–1.6 deg), and within a pick the near and far frames agree (r = 0.80). **The pallet heading of the labels carries a per-pick yaw error of about 1.3 deg (std).** The one alternative, a bias inherited through the fine-tuning lineage of runs 3–6, is excluded by a model trained from the public PointDiT weights for two epochs with no shared ancestry and a different split: it is far less accurate (16 cm overall) yet makes the same per-pick yaw errors as run 6, r = 0.89 over 177 picks. What these models share is the labels.

For the product this is decisive: the sideways error at distance is this heading error times the distance (section VII shows r = 0.98 between the two), so the lateral accuracy of the system is label-limited today. This reframes the error budget of section VI: the residual 4–5 cm "with GT mask and rotation" is largely label error, the network is probably closer to the physical pallet than the tables say, and the "GT pose" gains of the sequential estimator are measured against the same biased heading. The highest-value action is therefore on the data side: verify and correct the pallet heading in the pipeline's pallet fit (re-fit the face plane from the near-frame lidar, cross-check with the patch cloud), then regenerate the targets, which is cheap since the compact records only store the pose and the reconstruction follows, and re-evaluate. Sequence fusion cannot remove a label bias; with a consistent heading the warm-started network already delivers 5.5 cm overall and 8 cm at 6–8 m.

VIII. Status and current work (2026-10-20, evening)

Where PalletDiT stands. The deployed candidate is `runs/b256_prior2`: PointDiT-B/16 at 432x240 with the 6 + 4 channel head, fine-tuned in seven stages from the public B/256 weights, the last two with prior-conditioned training and the on-ray consistency loss, on 2,030 picks. On 201 picks no stage trained on, one forward pass per image gives, as medians:

face distance	lateral offset error	depth error	yaw error	3D origin error
0–2 m	3.0 cm	1.7 cm	0.8 deg	4.3 cm
2–3 m	4.3 cm	2.2 cm	1.0 deg	5.8 cm
3–4.5 m	6.7 cm	2.8 cm	0.9 deg	8.5 cm
4.5–6 m	9.2 cm	4.6 cm	0.9 deg	12.3 cm
6–8 m	13.4 cm	7.2 cm	1.0 deg	16.8 cm
8–12 m	15.6 cm	12.4 cm	0.9 deg	24.8 cm

What the product needs. The unit must align sideways with the pallet centreline from about 10 m onwards; the longitudinal distance can be informed by the lidar, which also makes the final assessment near the pallet. The headline metric is therefore the lateral offset error and the relative yaw by distance, and the table above is read column by column: depth is already adequate, lateral is what remains, and lateral is yaw times distance (r = 0.98).

What limits lateral accuracy today. The yaw error is a per-pick constant of about 1.3 deg (std) that every model shares, including one trained from the public weights with no common ancestry (r = 0.89): it is in the pallet-heading labels, not in the network. Until the heading in the lidar pallet fit is verified and corrected, the measured lateral error is a lower bound on the label error rather than a property of PalletDiT, and training cannot improve on it.

Yaw fusion along the approach (done, null result). The frame rotations were propagated with the odometry and blended with a weight of 0.2, 0.1 or 0.05 on each new frame, and every frame's votes re-projected with the fused rotation, on 60 clean picks (1,746 frames). The fused yaw error equals the per-

frame one (0.77–0.87 vs 0.77–0.79 deg) and so does the lateral error (4.9–5.2 vs 4.9 cm overall, identical per bin). Averaging the heading over the approach cannot help because the per-frame yaw error is the per-pick bias, not scatter. In lateral terms the warm start itself (own maps, own pose, t0 0.5) against single-frame on the same 60 picks reads 2.9 / 3.6 / 5.1 cm vs 3.3 / 3.9 / 6.1 cm at 0–2 / 2–3 / 3–4.5 m, and 12.3 / 19.3 vs 10.9 / 16.3 cm at 6–8 / 8–12 m: it helps near the pallet and hurts far away, where a stale prior from a worse frame dominates. A distance-dependent trust fixes this: scaling t0 from its full value at 3 m to zero at 6 m (using the previous frame's estimated distance), with t0 0.8 and pose weight 0.2, gives 2.9 / 3.6 / 5.3 / 7.5 / 10.9 / 16.3 cm across the six bins against 3.3 / 3.9 / 6.1 / 8.4 / 10.9 / 16.3 single-frame, jitter 4.1 to 3.3 cm, yaw 0.88 to 0.74 deg: better or equal at every distance. On all 201 clean picks (5,850 frames) the same configuration gives, single-frame to warm-started: lateral 3.3 / 4.2 / 6.7 / 9.8 cm to 3.1 / 4.0 / 6.0 / 9.4 cm in the four bins below 6 m (identical beyond, where the prior is off), yaw 0.90 to 0.84 deg, 3D origin 8.1 to 7.9 cm, depth 2.9 to 2.7 cm, with the pallet-centre bearing unchanged at 0.2–0.5 deg. The gain is consistent but modest, a few millimetres to seven millimetres sideways, because the heading it inherits carries the label bias; it is the sequential configuration to deploy, and its value grows with the heading accuracy.

Steering before the yaw is known. The unit can also steer on the pallet centre as seen from the camera, i.e. the bearing and the lateral position of the pallet origin in the camera frame, which do not depend on the heading estimate. For prior2 on the clean split these are far more accurate than the centreline offset:

face distance	lateral offset from the centreline (needs yaw)	pallet centre, lateral in the camera frame	bearing to the centre	depth to the centre
0–2 m	3.0 cm	1.4 cm	0.48 deg	1.6 cm
2–3 m	4.3 cm	1.8 cm	0.42 deg	2.2 cm
3–4.5 m	6.7 cm	1.8 cm	0.28 deg	2.6 cm
4.5–6 m	9.2 cm	1.9 cm	0.19 deg	4.2 cm
6–8 m	13.4 cm	2.6 cm	0.21 deg	6.9 cm
8–12 m	15.6 cm	3.1 cm	0.20 deg	11.2 cm

PalletDiT locates the pallet centre to 2–3 cm sideways and 0.2–0.5 deg of bearing at every distance, including 10 m. What it cannot yet give to better than ~1 deg is the pallet's heading, and only the heading is label-limited. For the approach this suggests steering on the centre bearing from afar, and aligning to the centreline (heading) as the lidar takes over near the pallet, until the heading labels are fixed and the yaw can be trained and evaluated properly.

Next, in order.

1. Data side: verify and correct the pallet heading in the SensorUnit pallet fit (worst picks listed in `docs/yaw_bias_per_pick_2026-10-20.csv`), regenerate the poses, re-evaluate PalletDiT on the same 201 picks. If the labels were the problem, the lateral numbers above should drop substantially without retraining.
2. Yaw-focused training once the labels are clean: more weight on the phi channel and a within-pick heading-consistency term (the pallet does not move during an approach).
3. The sequential estimator in deployment form: warm start from the fused pose, odometry noise model, cost of one extra step.
4. Far field: resolution and data diversity, which only matter beyond 6 m.
5. Export: the DINOv3 encoder already runs on the unit; the ViT is plain attention.

1. PointDiT: H. Xu et al., *PointDiT: Pixel-space Diffusion Transformers for Monocular Geometry Estimation*, 2026 (arXiv:2607.02515).
2. DINOv3: Meta AI, 2025.
3. Data pipeline: `~/Anantak/SensorUnit/docs/PALLET_DATA_BUILDING_PROCESS.md` (SensorUnit, branch THD).